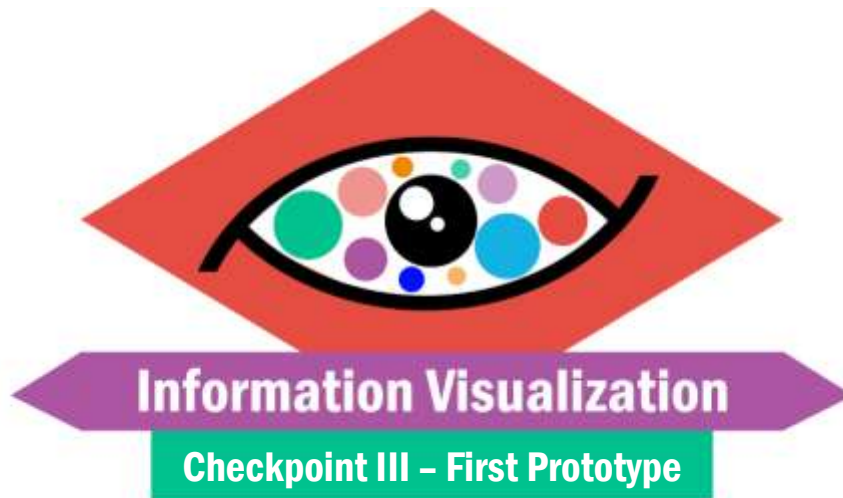




Objective

At the end of this checkpoint, you have a functional version of your visualization mainly consisting of creating the layout and implementing at least one of the idioms you have selected on Checkpoint II.

Requirements

Create your functional visualization prototype based on the feedback you received regarding your sketch (Checkpoint II). In this functional prototype, **focus on the structure of the visualization interface**: make sure you organize the visual space into different areas according to each idiom you have previously selected. We also want to know the code structure, e.g., modules, you use to implement the prototype. Create a simple flow chart showcasing how the data transfers from module to module in the prototype is enough to support your explanation.

For this assignment, **you must fully implement at least one of the idioms**, perfectly matching the final aesthetics and functionality you have planned for your visualization. You must use the actual data you have gathered (Checkpoint I). If you implement more than one idiom, **make sure that your idioms are interactive**. They need to provide the means for some interaction (e.g., a chord diagram must highlight relationships for items selected, a bar chart must show adequate behavior when moving over a bar, etc.). Static images are not going to get a good grade. As we already stated in CPII, we will assume explicit interactions for specific visual idioms.

Then, keep in mind that **your views must be linked in meaningful ways**, so think about how the views share the data and how each view can inform the remaining interactions taking place (e.g., when you select an item in view X, how will you let views W and Z know so that they can choose the same item?). We want to see the **integration structure/function/package** that makes everything work together. You must implement an extensible prototype (i.e., when you have all the idioms together, the mechanisms you implement here should still work to link all of them). If you only implemented one algorithm, you must use abstraction and proper programming skills to have a modular implementation to quickly receive the new visual idioms in the following checkpoints. Describe how your prototype integrates the visualizations.

Also, **the idioms need to look coherent** (e.g., similar text fonts or colors). We want a cohesive system, not a patchwork quilt of visualization techniques. In addition, remember that even if you implement only one idiom, you must leave space for the others. So, where they will ultimately appear, **include a sketch image (you can take these from CPII) of the missing idioms**. Use the expected size/aspect ratio. In short, we expect to see the final prototype's layout.

Finally, but important, **you are only allowed to use D3.js**. No other frameworks are permitted. This means do not use React, Angular, Node.js, or any similar tools. If you do, your checkpoint will receive a **grade of 0**. You may use Python (and Python alone) for the backend, if necessary, but your frontend must be built using only plain JavaScript, HTML, and CSS. No TypeScript, no transpiling, no SASS, etc. Also keep in mind your prototype must be stand-alone so if you use any JS libraries, etc. make sure you have a local copy. Rule-of-Thumb: if it works without an internet connection, you're probably fine.

Deliverables

Create a **video with a maximum of 3 minutes** and submit it online, until two days before your class (ex: classes on Friday must submit until Wednesday at the end of day). Use either the h264 or h265 codec, and a modern container (mp4, mkv, etc.). Weird stuff we cannot play with the default installs of Windows/MacOS/Debian will not be graded. Name your video '**CPIII-<group number>.<extension>**' (the extension will depend on the container format you choose).

Submit it online, **inside a zip file including both the video and the code, in Moodle**.

The **video** should describe and illustrate:

- The **code architecture** (NO CODE) you have created to load and (pre)process the data, to make the charts (and their interactivity), and to handle the integration.
- The **layout of your visualization**. Include at least one shot of the complete interface (yes, even the idioms you have not implemented, in which case use screenshots from Checkpoint II).
- The **data (pre)processing** you must do before the data is ready to be loaded to the charts. We are not talking about the data processing from CPI but all the remaining process you must do for each visual idiom.
- The idioms you have implemented (**chart visualization**) and their functionality (**chart interactivity**). You must describe them and explain the interactivity supported by those idioms.
- The **chart integration** techniques used and **why adding more charts is easy**. How are the views linked? How does that mechanism work/will work even when you have more idioms to link?

If there are any changes from Checkpoints I and II, describe them and justify why you made those changes using the visualization principles learned in class and **VisDeck card spreads**. You will need to defend those spreads to validate the changes to your design. So, it is not enough to just choose an idiom you think easy or cool. The spread must match it to your questions and dataset. **Changes not justified will be penalized!**

Your prototype must **include the dataset**. You should deliver in a plug-and-play fashion: we run a local server, and the prototype is ready to be tested.

Name your deliverable as '**CPIII-<group number>.zip**'. For instance, 'CPIII-02.zip'. The zip should contain the report and your code. Make sure your PDF file follows the same name nomenclature.

Penalties

- Video over 3 min long: **1 grade point penalty per extra 10sec**.
- Video uploaded after the deadline: **0.5 grade points penalty per hour of delay**.
- Incorrect file type or name: **0.5 grade points penalty per file or name**.
- Video file and ZIP names are different: **0.5 grade points penalty**.
- Incorrect file type or name: **0.5 grade points penalty per file or name**.
- Zip with video and code uploaded after the deadline: **0.5 grade points per hour of delay**.
- No prototype: **You will be graded 0 in this checkpoint**.

Tasks to perform during the lab

Show your working prototype to the professor for **feedback**.

The grade **will be made known one week later** (see below). If you are not receiving feedback, then you must be peer assessing your colleagues' submissions.

Grading

Your work will be graded according to the following parameters:

- **Video (5%):** within allotted time, easy to understand, follows suggested structure
- **Prototype architecture (15%):** describe and illustrate the code architecture to produce the prototype.
- **Dashboard layout (15%):** present and depict the work done so far according to your Checkpoint II.
- **Additional data processing and technique changes (5%):** describe your additional data processing techniques to achieve the result. If you do not have additional data processing, explain why. Also describe changes to the visual idioms and justify them with VisDeck.
- **Chart visualization (20%):** show the actual visualization plotting real data, with a polished look-and-feel.
- **Chart interaction (20%):** present the code modules used to achieve chart interaction.
- **Chart integration (15%):** describe the integration architecture between charts.
- **Peer assessment (5%):** grade your peers' submissions in the class.

An important note on grading for this and all other Checkpoints: always *justify* your choices, based on the basic principles you have learned in this course (adequacy of channels to data types, human perception, etc.). Don't just describe what it is, but especially *why* it is as it is.